Introduction to Artificial Intelligent (AI)

2. Problem solving by Searching



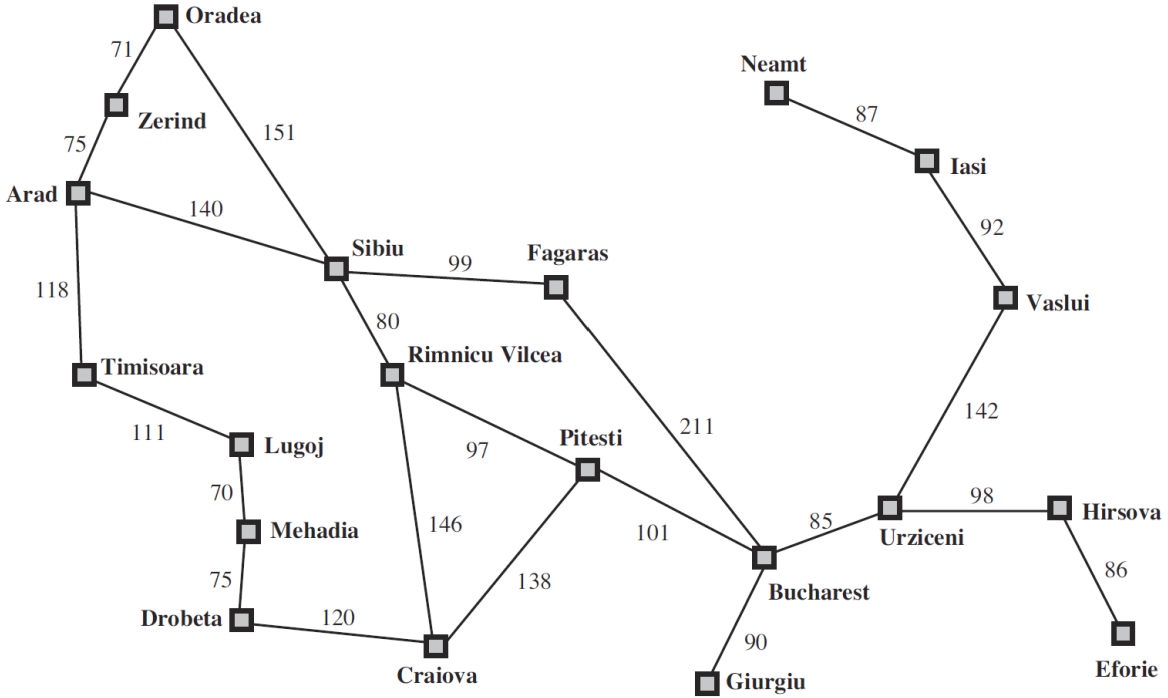
Define a problem

A **problem** can be defined by **5 components**:

- **Initial state**
- **Possible Actions**: a set of actions that can be executed in each state
- **Transition (successor) function**: returns the state that results from doing an action, but does not tell which action to use
- **Goal test**: check if the current state is goal state
- **Cost**: a cost function that reflects its own performance measure

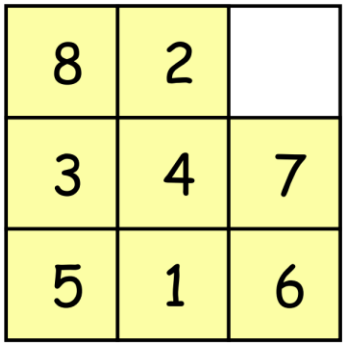
Define a problem

Problem: find path from Arad to Bucharest

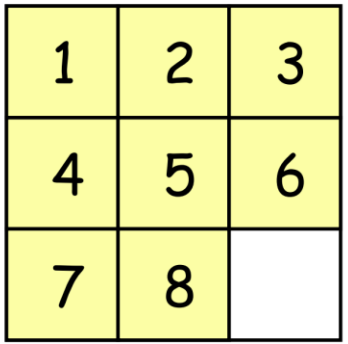


Define a problem

Problem: 8 - Puzzle



Initial state



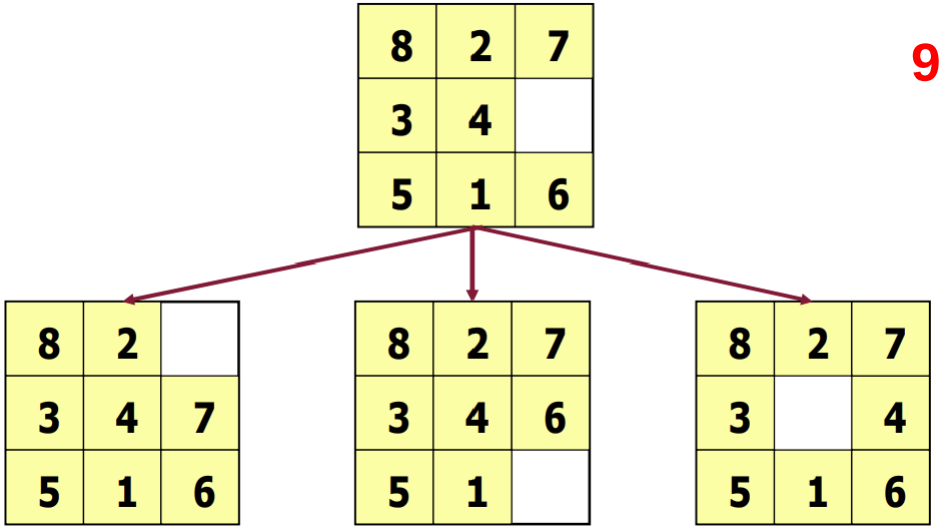
Goal state

Define a problem

Problem: 8 - Puzzle

How many states 8-puzzle has?

9!/2 = 181,440 states



Transition (successor) function returns state/alternatives.

Define a problem

Problem: 15 - Puzzle

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

?

1	2	3	4
5	6	7	8
9	10	11	12
13	15	14	

In 1878, Sam Loyd offered **$1,000** to the first person who would solve the following problem

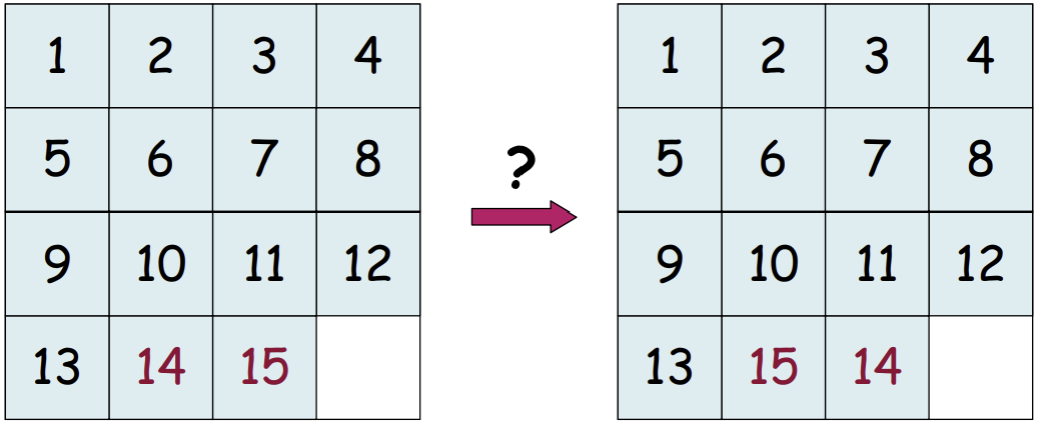
But no one ever won the prize!!



Sam Loyd (1841-1911)
America's greatest puzzle-expert

Define a problem

Problem: 15 - Puzzle



Search methods are used to explore and find the optimal alternatives

15-puzzle has 16!/2 ~ 1.04 x 10^{13} states
→ too large to store and would take too long to learn

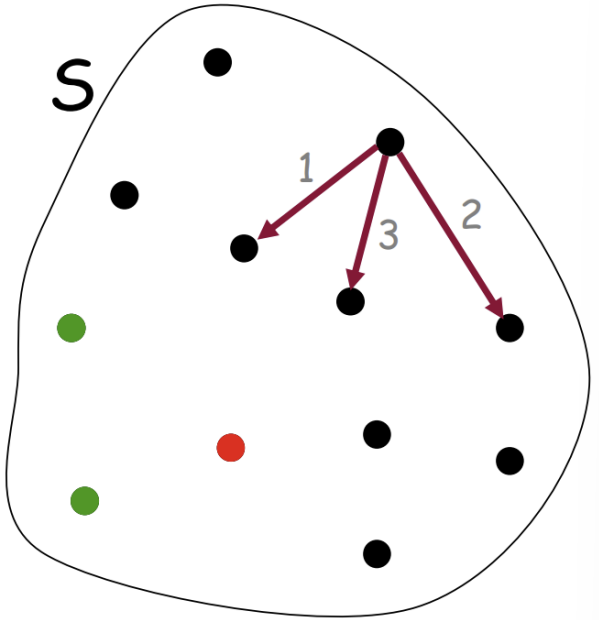
Define a problem as a search problem

Search problem's components:

- **Initial state**
- **State space S**
- **Successor function**:

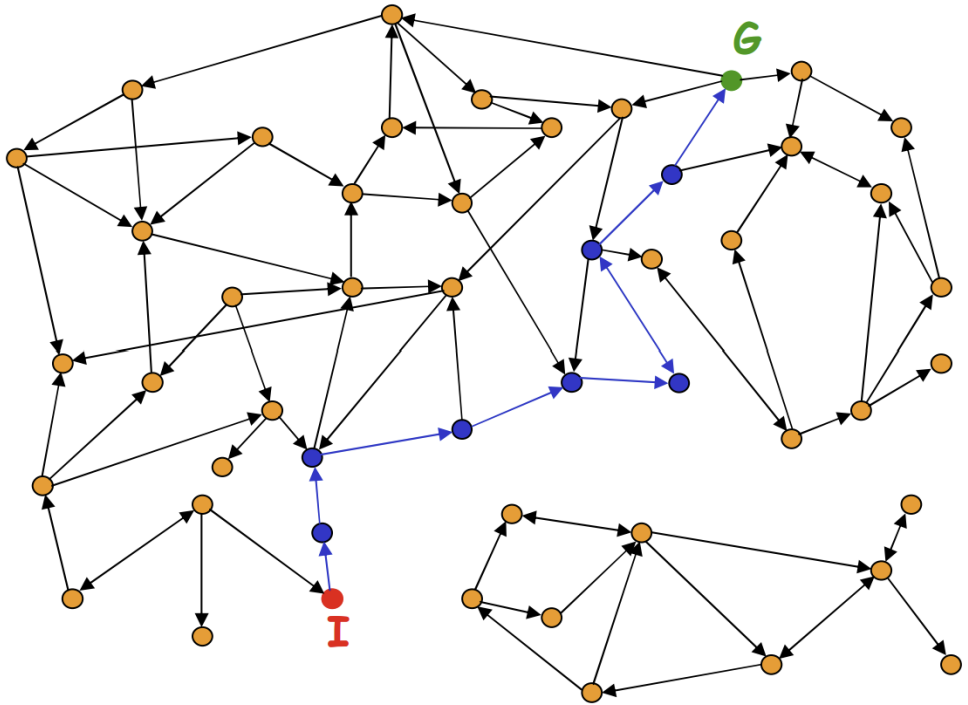
$$x \in S \rightarrow \text{SUCCESSORS}(x) \in 2^S$$

- **Goal test**: $x \in S \rightarrow$ GOAL?(x) =T or F
- **Arc Cost**



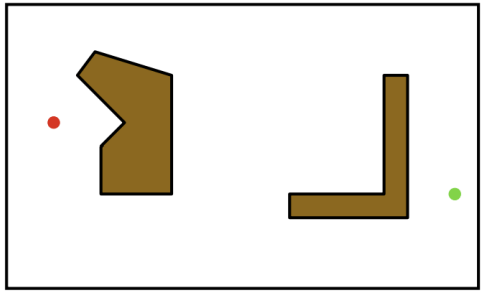
State Graph

- Each state is represent by a **node**
- An **arc** connects a node **s** to node **s'** if s' ∈ SUCCESSORS(s)
- A **solution** is a path connecting the initial node to a goal node
- The **cost** is the sum of the arc costs along a path
- An **optimal solution** is a solution path of minimum cost
- There might be no solution !

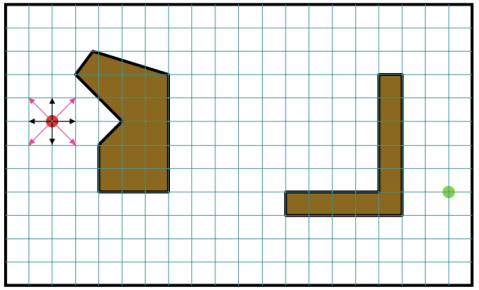


Problem formulation

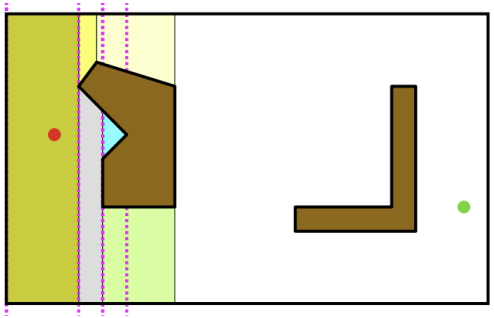
A **problem** can be formulated in different ways.



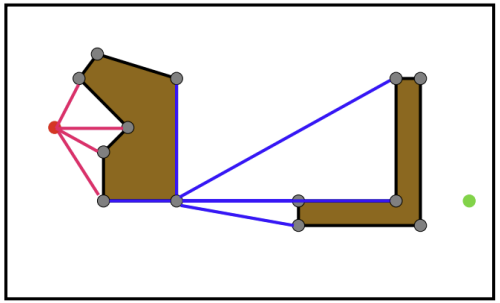
Problem: path finding



#1

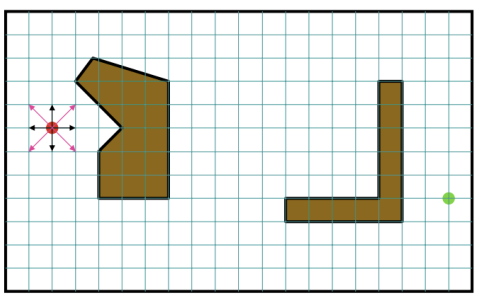


#2

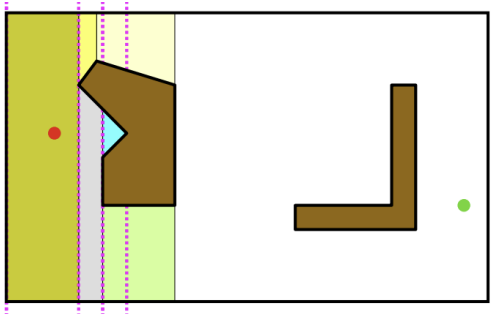


#3

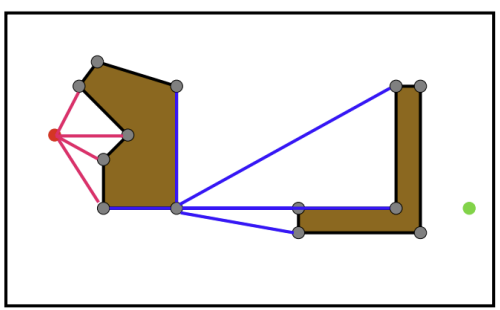
Problem formulation



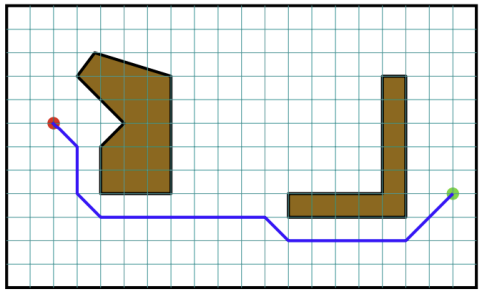
#1

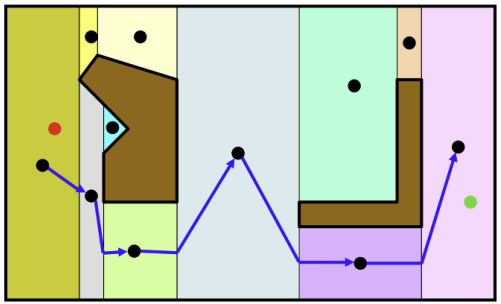


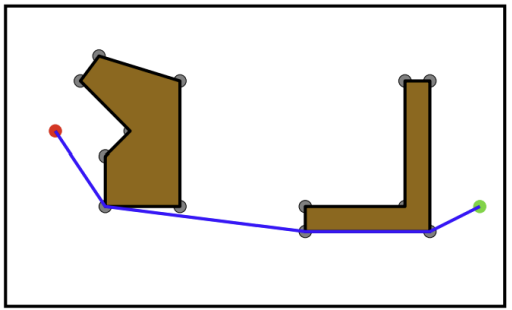
#2



#3







State space

How big is the state space of the $(n^2 - 1)$-puzzle?

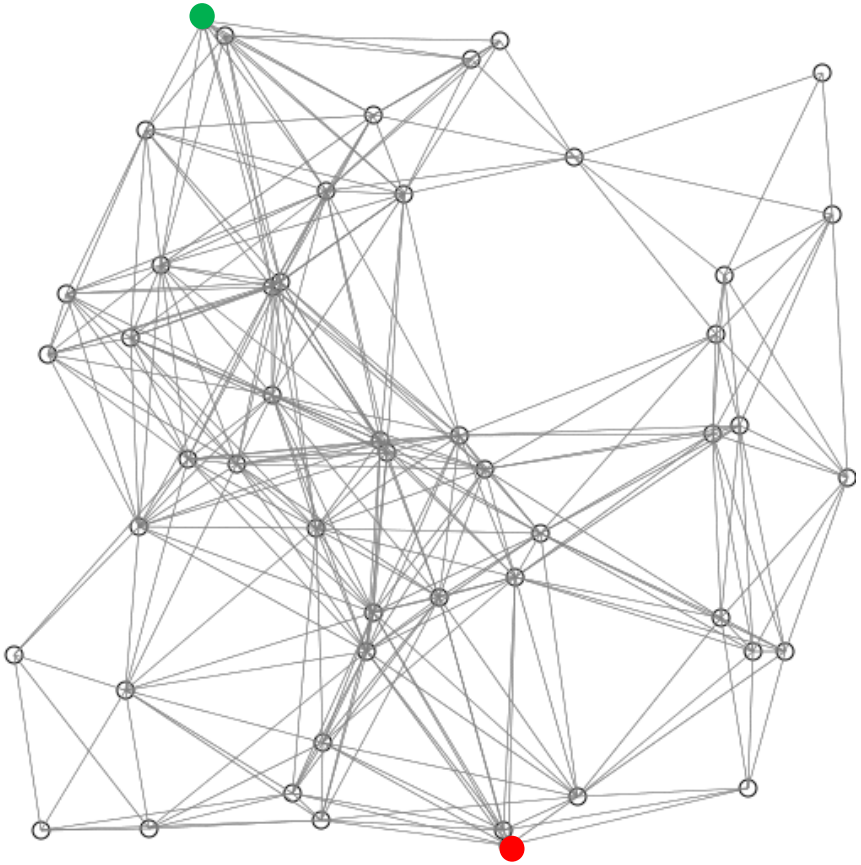
- 8-puzzle → 9! = 362,880 states → 0.036s
- 15-puzzle → 16! ~ 2.09 x 10^{13} states → 55 hours
- 24-puzzle → 25! ~ 10^{25} states → 10^9 years

If a computer can do 100 million state/sec

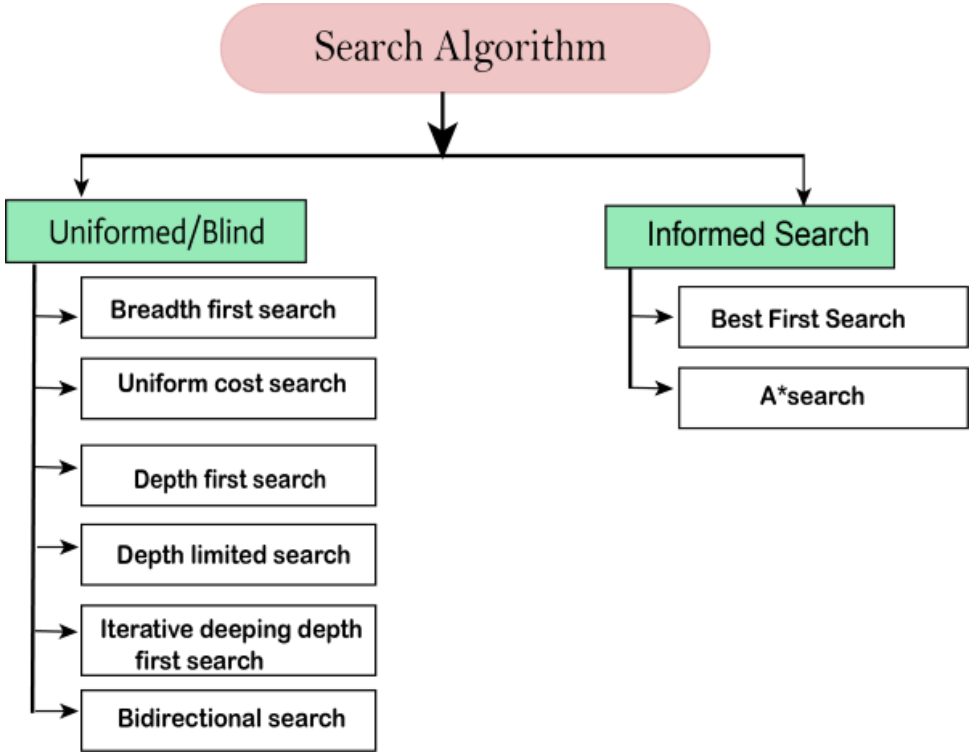
It is often not feasible (or too expensive) to build a complete state graph

A problem solver must construct a solution by exploring a small portion of the graph

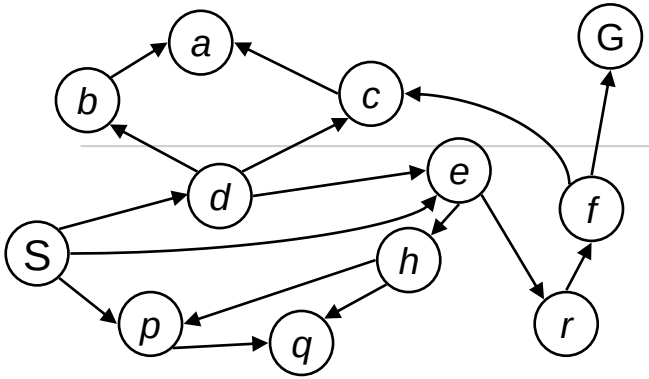
Searching the State Space



Search Algorithms

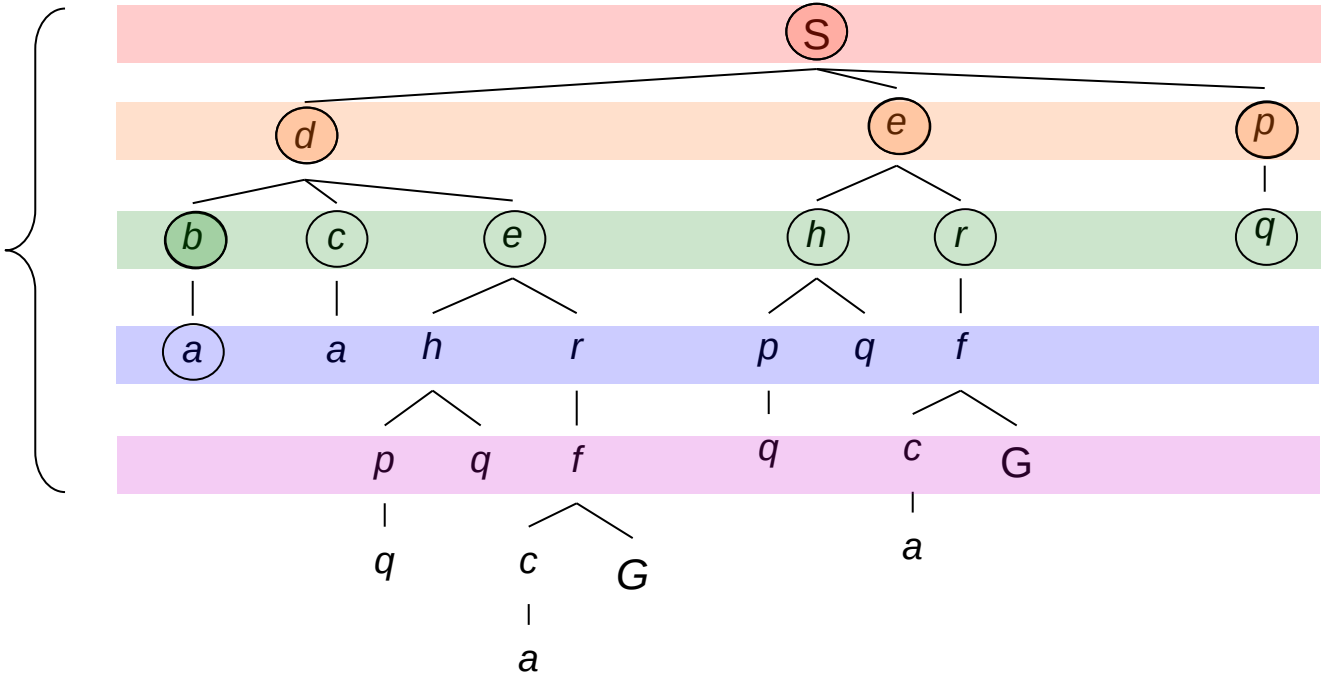


Blind search



Breath First Search

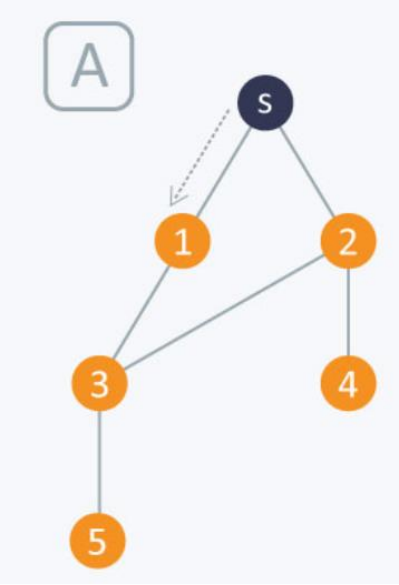
Strategy: expand a shallowest node first



Blind search

Breath First Search

Strategy: expand a shallowest node first



Visited

S						

Queue

1	2					

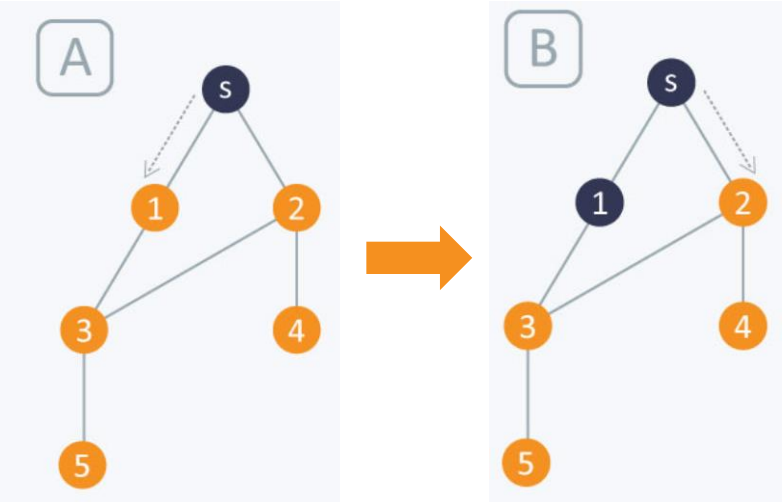
First *Last*

First In First Out (FIFO)

Blind search

Breath First Search

Strategy: expand a shallowest node first



Visited

S	1					

Queue

2	3					

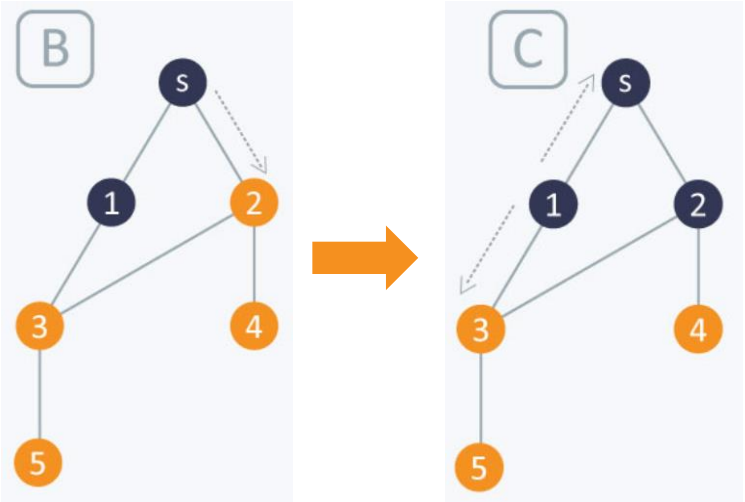
First *Last*

First In First Out (FIFO)

Blind search

Breath First Search

Strategy: expand a shallowest node first



Visited

S	1	2				

Queue

3	4					

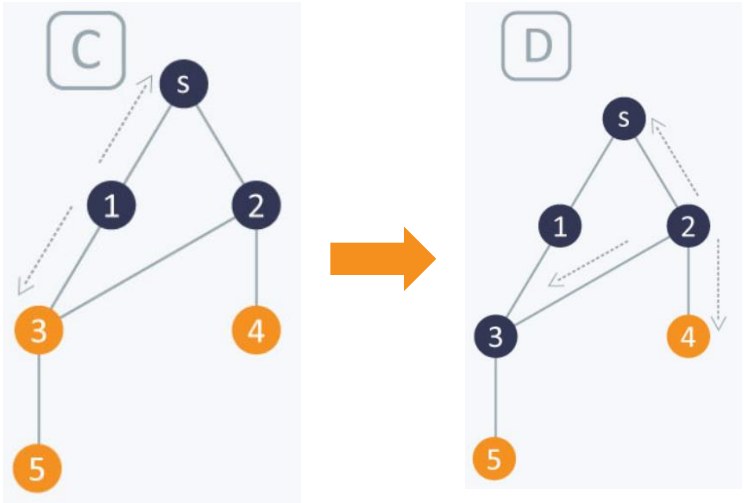
First *Last*

First In First Out (FIFO)

Blind search

Breath First Search

Strategy: expand a shallowest node first



Visited

S	1	2				

Queue

4	5					

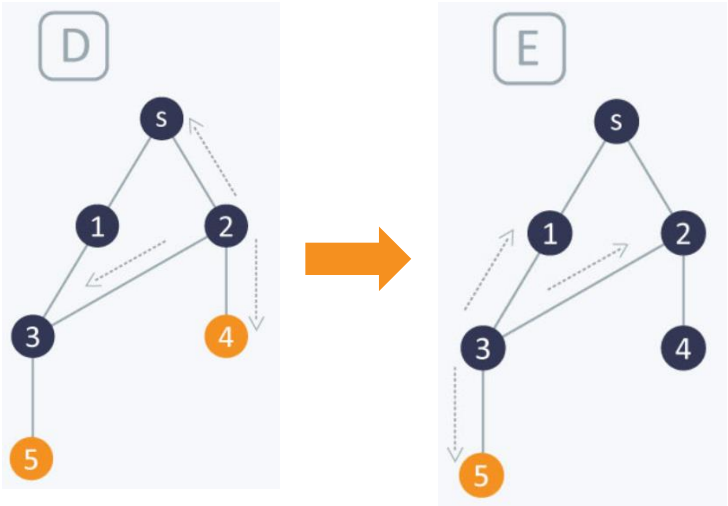
First *Last*

First In First Out
(FIFO)

Blind search

Breath First Search

Strategy: expand a shallowest node first



Visited

S	1	2	3	4		

Queue

5						

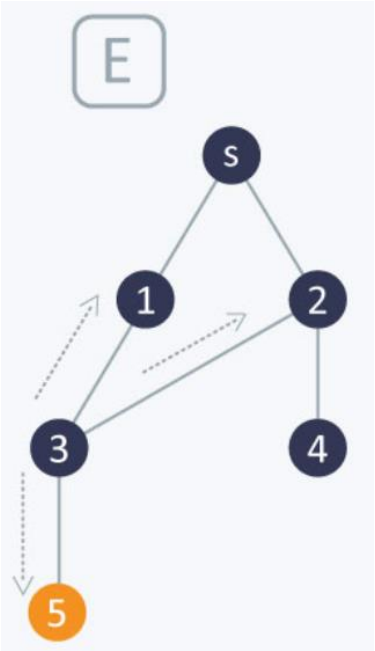
First *Last*

First In First Out
(FIFO)

Blind search

Breath First Search

Strategy: expand a shallowest node first



Visited

S	1	2	3	4	5	

Queue

Shortest Path

S	2	3	5			

Blind search

Breath First Search

Strategy: expand a shallowest node first

```
import collections

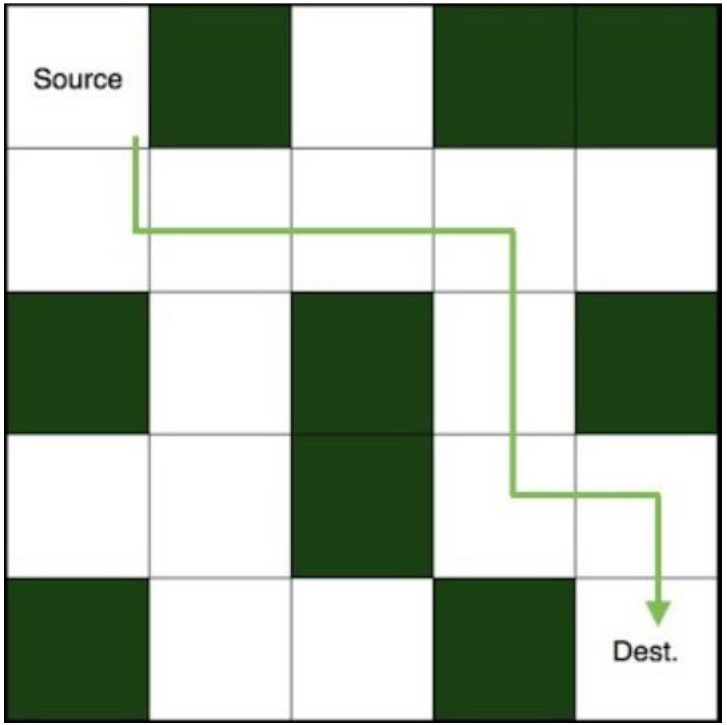
# BFS algorithm
def bfs(graph, source):
    visited, queue = set(), collections.deque([source])
    visited.add(source)
    while queue:
        # Dequeue first node from queue
        node = queue.popleft()
        print(str(node) + " ", end="")
        # If not visited, mark it as visited, and
        # enqueue it
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
graph = {"S": ["1", "2"], "1": ["3"], "2": ["3", "4"],
         "3": ["5"], "4":["3"] , "5":["4"]}
bfs(graph, "S")
```

Blind search

Breath First Search

Strategy: expand a shallowest node first

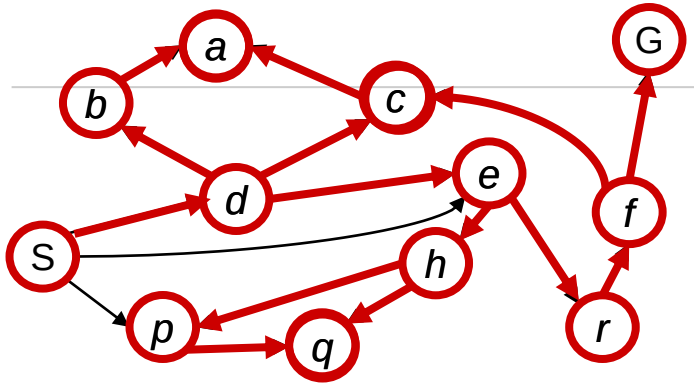
Exercise: Rat in a Maze Problem

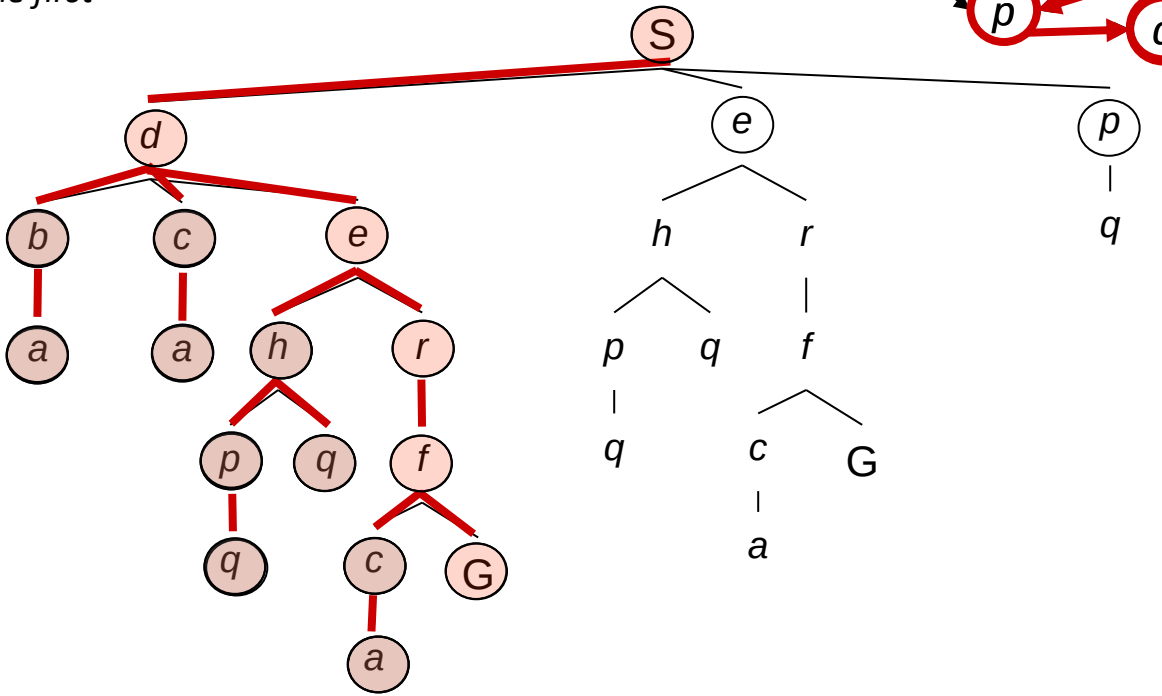


Blind search

Depth First Search

Strategy: expand a deepest node first

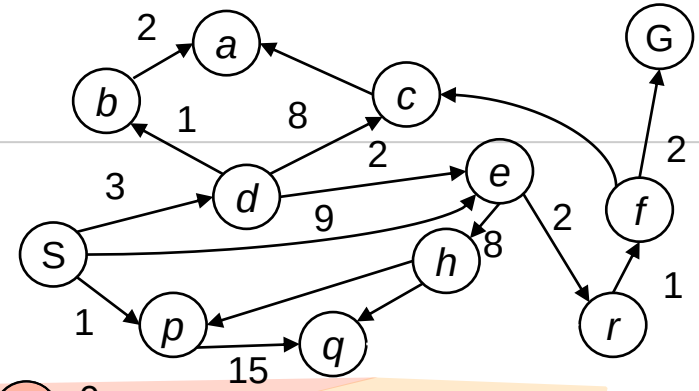


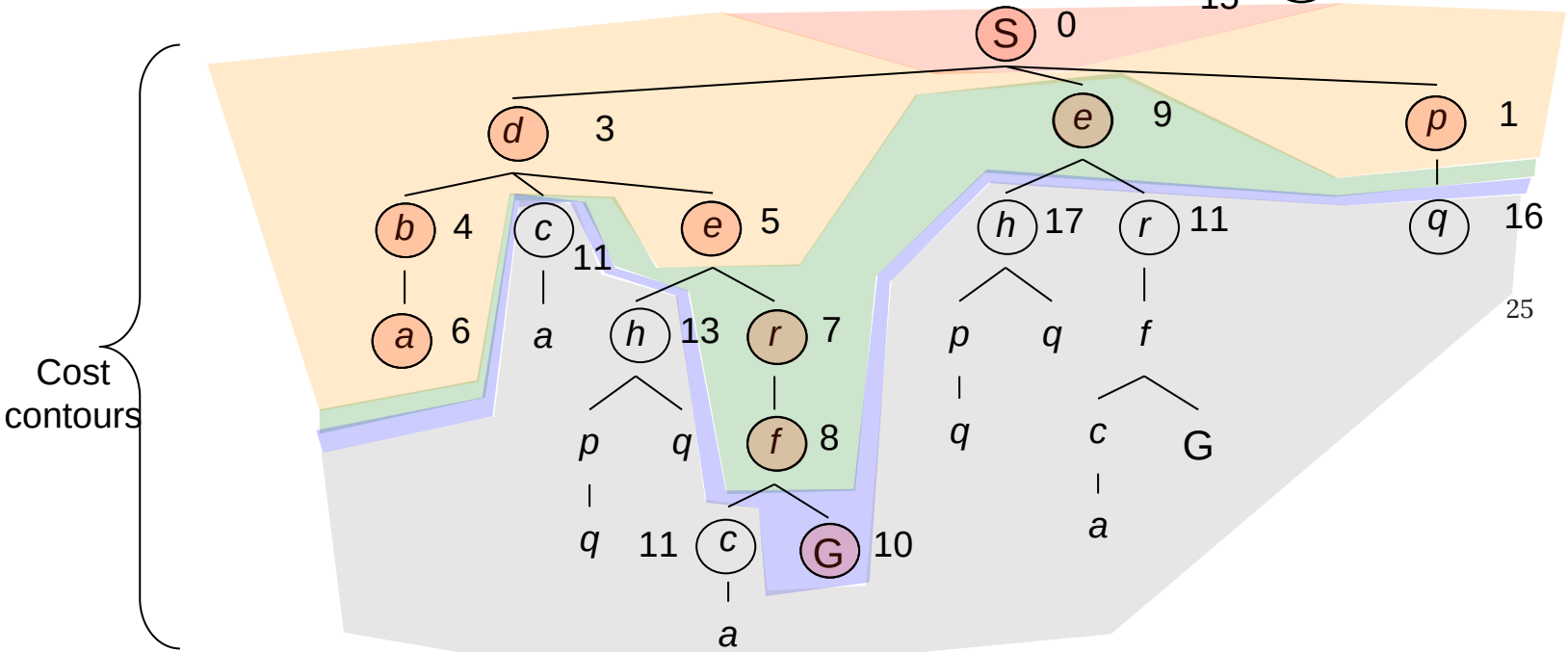


Blind search

Uniform Cost Search

Strategy: expand a cheapest node first:





Heuristic search

Heuristic function h(n) is a function used to estimates how close a state is to the goal. h(n) is always positive

Heuristic search find node/path based on their heuristic value h(n).

Heuristic search uses 2 lists:

- OPEN: nodes which have yet been expanded
- CLOSED: nodes which have already expanded

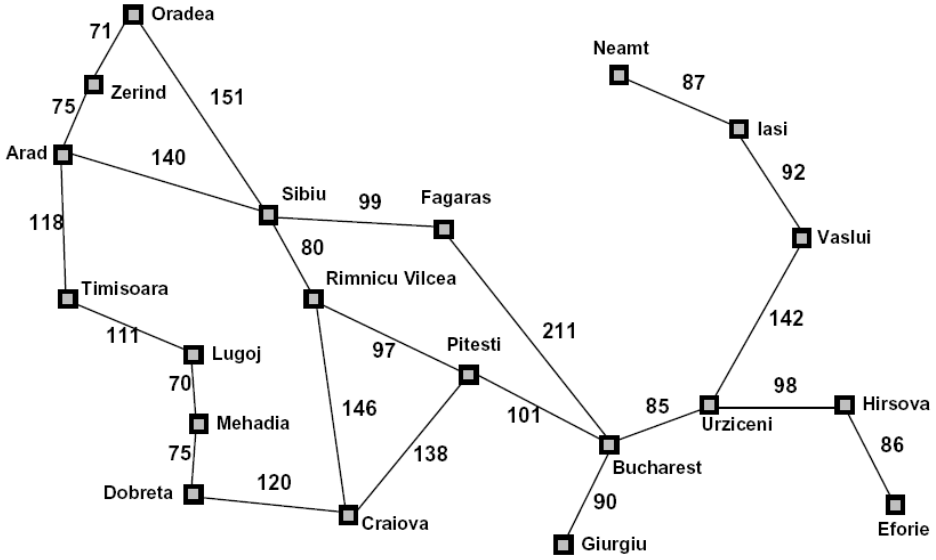
On **each iteration**, node n with the lowest heuristic value is expanded and generates all its successors, after that n is placed to the closed list.

2 main heuristic search algorithms:

- Greedy search
- A* search

Heuristic search

Greedy Search : selects the path which appears best at that moment by combination of DFS and BFS.

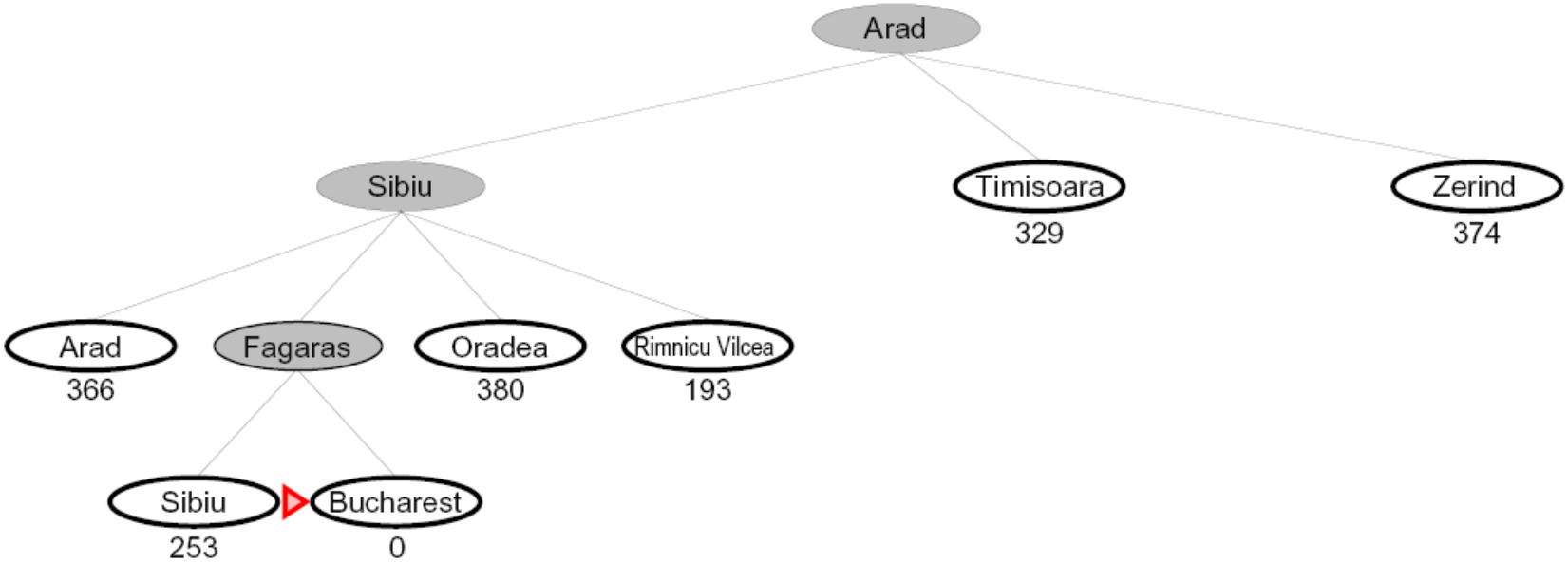


Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

h(x)

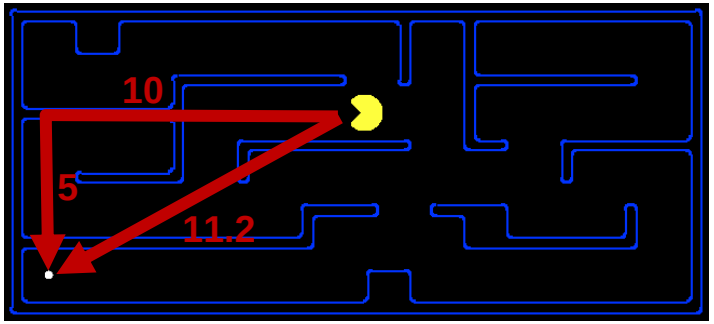
Heuristic search

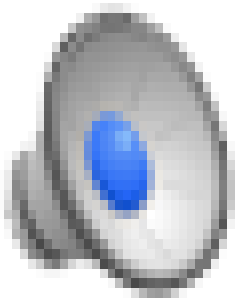
Greedy Search



Heuristic search

Greedy Search



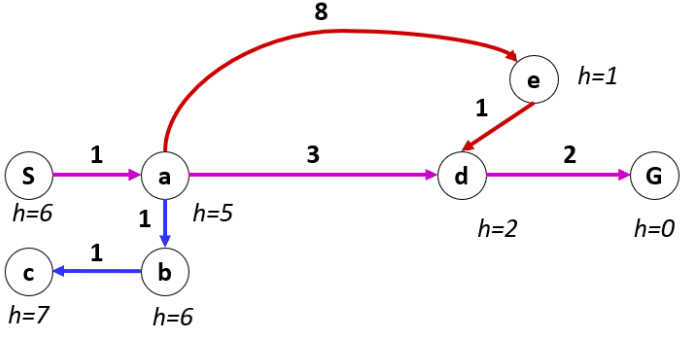


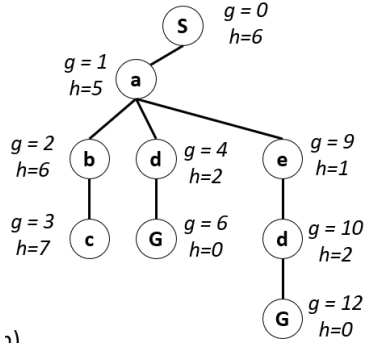
Heuristic search

A* Search : find the shortest path by combine heuristic cost and cost to reach node n from start state

$$f(n) = g(n) + h(n)$$

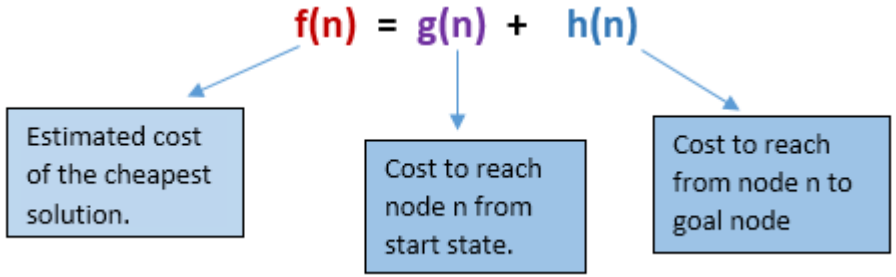
- $h(n)$: Estimated cost of the cheapest solution.
- $g(n)$: Cost to reach node n from start state. — Uniform-cost search
- $h(n)$: Cost to reach from node n to goal node — Greedy search

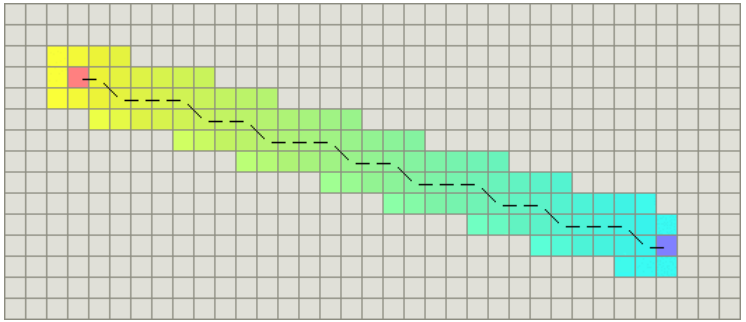




Heuristic search

A* Search : find the shortest path by combine heuristic cost and cost to reach node n from start state





```
function heuristic(node) =
    dx = abs(node.x - goal.x)
    dy = abs(node.y - goal.y)
    return D * (dx + dy) + (D2 - 2 * D) * min(dx, dy)
```

http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html

Heuristic search



Greedy



Uniform Cost



A*

Heuristic search

A* Search

